You said:

Lab_4_VibeCodin...
PDF

this is the context file:
github link: https://github.com/SETAPESU26/20_mastermind
give me cmds to pull and test the game, before , tell me what the project is about as well, list out the delivarables

ChatGPT said:

Lab 4 — VibeCoding: Mastermind

Your assignment is to take the assigned Python game, **identify and fix its existing bugs, implement the four features listed in its README, test the changes, and push your completed work to your own GitHub repository.**

I checked the repository's README and your uploaded Lab 4 handout. The README defines the game-specific requirements; the handout defines the lab workflow and submission requirements. GitHub Lab_4_VibeCoding_Student_handout.pdf

1. What is the project about?

Mastermind — a terminal-based code-breaking game



The computer generates a secret code, and you try to guess it. After each valid guess, the game provides feedback indicating how many symbols are correct and how many are in the wrong position.

For example, suppose the secret code is `1 2 3 4` and your guess is `1 3 2 5`.

- **Exact matches:** `1` is correct and in the correct position → 1 exact match.
- **Partial matches:** `2` and `3` occur in the secret code but are in different positions → 2 partial matches.
- **No match:** `5` does not occur in the secret code.

The important programming challenge is handling repeated symbols correctly. One occurrence in the secret code must not be counted multiple times when calculating feedback.

The repository contains four main files: `main.py`, `game.py`, `logic.py`, and `requirements.txt`. GitHub

2. Deliverables — what you must complete

There are four coding tasks and three main submission items.

Coding deliverables

Task 1 — Fix duplicate-aware feedback

Calculate exact and partial matches correctly, resolving exact matches first. No code occurrence can contribute more than once.

Task 2 — Complete the game lifecycle

Display clear win/loss outcomes, preserve guess history, enforce the guess limit, and prevent changes after the game ends.

Task 3 — Add difficulty modes

Vary code length, available symbols, and allowed guesses by difficulty. Feedback must work for every supported configuration.

Task 4 — Improve history and input validation

Display readable guess history. Reject malformed guesses without consuming a turn, and produce feedback only for accepted guesses.

Submission deliverables

Deliverable	What to submit
1. Before video	10-second recording showing the original bug or broken behaviour
2. After video	10-second recording demonstrating the fix and new features
3. Updated code	Completed project in your personal GitHub repository
4. LLM conversation	Complete ChatGPT/LLM conversation history, as a link and/or exported document

The repository README explicitly lists the two videos and complete LLM chat-history link. Your lab handout additionally asks you to upload the updated code and an exported chat history as a document or PDF under your lab repository's `Lab–4` folder. To satisfy both, include **all four items**. GitHub Lab_4_VibeCoding_Student_handout.pdf

Other important requirements:

- Make separate Git commits for individual tasks.
- Work individually.
- Do not open a pull request to the original `SETAPESU26` repository.
- Do not add unnecessary dependencies or persistent storage such as CSV, JSON or SQLite.
- Test the specified edge cases, not just whether the game starts.

3. Commands to pull, run and test the game on your Mac

These commands are for your MacBook using Terminal.

Step 1 — Fork the repository

Open the assigned repository: 20_mastermind on GitHub ↗.

1. Sign in to GitHub.
2. Click **Fork** and create a copy under your personal GitHub account.
3. Use your fork for your changes. Do not push to the original `SETAPESU26` repository.

Step 2 — Clone your fork

Replace `YOUR_GITHUB_USERNAME` with your actual GitHub username.

```
cd ~/Desktop

git clone https://github.com/YOUR_GITHUB_USERNAME/20_mastermind.git

cd 20_mastermind
```

If you already cloned the repository previously, use this instead to pull the latest changes:

```
cd ~/Desktop/20_mastermind

git pull
```

Note: `git pull` only updates an existing local repository. It does not clone a repository for the first time.

Step 3 — Inspect the project

Before changing anything, inspect the existing code and requirements.

```
pwd
git status
git remote -v
ls -la

cat README.md
cat requirements.txt

sed -n '1,240p' main.py
sed -n '1,260p' game.py
sed -n '1,260p' logic.py
```

You should see the four project files specified in the README. Read the task requirements carefully before asking an AI coding tool to make changes.

Step 4 — Set up Python and run the original game

Check your Python installation:

```
python3 --version
```

Create a virtual environment and activate it:

```
python3 -m venv .venv
source .venv/bin/activate

python -m pip install --upgrade pip
```

The README gives `python main.py` as the launch command and does not specify any mandatory installation step. Check `requirements.txt` first. If it lists dependencies, install them:

```
python -m pip install -r requirements.txt
```

Now launch the game:

```
python main.py
```

If the game starts successfully, play it and deliberately test guesses containing repeated symbols. For example, compare a secret code conceptually equivalent to `1 2 3 4` with a guess such as `1 1 1 1`.

Before modifying any code, record the required 10-second video showing the original behaviour or bug. Save it outside the source files, for example on your Desktop.

Step 5 — Test the original defect

The main bug to investigate is whether repeated symbols are counted more than once in the feedback.

Use this reference table to understand the expected behaviour:

Secret code	Guess	Exact	Partial
`1 2 3 4`	`1 2 3 4`	4	0
`1 2 3 4`	`4 3 2 1`	0	4
`1 2 3 4`	`1 3 2 5`	1	2
`1 1 2 2`	`1 2 1 3`	1	2
`1 2 3 4`	`5 5 5 5`	0	0

These cases illustrate the matching rules; they are not a substitute for testing the actual implementation. In particular, repeated occurrences must be matched one-to-one.

Step 6 — Make and test the changes task by task

Use your coding assistant to fix and implement the requirements in the README. Avoid replacing the whole project with an unrelated implementation.

After each task, run:

```
python main.py
git diff
git status
```

Check the changes before committing them. If the code is broken, fix it before proceeding to the next task.

Once the four tasks are complete, verify the following cases:

- [ ] Exact matches
- [ ] Repeated symbols in both the secret and guess
- [ ] No matching symbols
- [ ] Mixed exact and partial matches
- [ ] Every difficulty setting
- [ ] Invalid input does not consume a turn
- [ ] Winning on the final allowed turn
- [ ] Losing after exhausting guesses
- [ ] Quitting the game
- [ ] No state changes after the game ends

These are the test cases specified in the repository README. GitHub

Step 7 — Commit each task separately

Once each task has been implemented and checked, make separate commits.

```
git add logic.py
git commit -m "Fix duplicate-aware feedback"

git add game.py main.py
git commit -m "Complete game lifecycle"

git add main.py game.py logic.py
git commit -m "Add difficulty modes"
```

```
git add main.py game.py logic.py
git commit -m "Improve guess history and input validation"
```

These commands assume the relevant changes are in the named files. If your changes affect different files, adjust `git add` accordingly. Do not create commits for tasks that are not yet completed and tested.

Step 8 — Push to your personal GitHub repository

Confirm that `origin` points to your personal fork:

```
git remote -v
```

If it points to your fork, push the branch:

```
git branch --show-current
git push -u origin main
```

If your current branch is not `main`, push the branch name reported by the previous command instead.

If you originally cloned the assigned repository directly, change the `origin` URL to your fork before pushing:

```
git remote set-url origin
https://github.com/YOUR_GITHUB_USERNAME/20_mastermind.git

git remote -v
git push -u origin main
```

Do not push your changes to `SETAPESU26/20_mastermind`.

Step 9 — Record the final video and prepare submission

After all tasks are completed and tested:

1. Record another 10-second video showing the fixed game and new features.

2. Export your complete AI conversation history as a PDF or document, and retain its shareable conversation link if available.
3. Place the videos, updated code and conversation export in the `Lab–4` folder of your lab submission repository.
4. Verify that your personal GitHub repository contains the latest commits.

4. Recommended execution order

Your lab checklist 0/10 complete

- [ ] Fork and clone the assigned repository
- [ ] Inspect the README and existing Python files
- [ ] Run the original game and record the bug
- [ ] Fix duplicate-aware feedback
- [ ] Complete win/loss and game-state handling
- [ ] Implement difficulty modes
- [ ] Improve history and input validation
- [ ] Test all required edge cases
- [ ] Commit each task separately and push
- [ ] Record the final video and prepare Lab-4 deliverables

Important: The lab is designed to be completed within 60 minutes, so prioritize reproducing the original defect, implementing the four required tasks, and testing the required cases. Avoid unrelated UI redesigns or extra features.

You said:

okay, what all tasks must i try with?, please give me the updated code and test it out again, and i also need to export this chat

Worked for 20s

Analyzed

ChatGPT said:

1. The exact tasks you need to complete with your AI coding assistant

The repository README confirms these four tasks. SETAPESU26/...

Use one prompt per task so that your work and Git history remain separate.

1 **Fix duplicate-aware feedback**

File: `logic.py`

Prompt your coding assistant to fix exact and partial match counting. Resolve exact matches first and ensure each code occurrence is counted at most once.

Copy prompt

2 **Complete the game lifecycle**

File: `game.py`

Handle wins, losses, the guess limit, guess history, quitting, and preventing further gameplay once the game has ended.

Copy prompt

3 **Add difficulty modes**

Files: `main.py`, `game.py`, `logic.py` as needed

Provide easy, medium, and hard modes with different code lengths, symbol ranges, and allowed guesses.

Copy prompt

4 **Improve history and input validation**

Files: `game.py`, `main.py` as needed

Reject invalid guesses without consuming turns and display readable history.

Copy prompt

Capture your **before video before making any changes**. Commit each task separately after implementing and testing it.

2. Download the updated code

I prepared an updated implementation that retains the original Python project structure and adds automated tests.

Mastermind — updated project

ZIP · Python source files and automated tests

Download updated code ↗

Automated test report

PDF · Test results and execution commands

Download test report ↗

What changed?

- `logic.py` — fixes duplicate-aware feedback.
- `game.py` — handles wins, losses, quitting and game history.
- `main.py` — lets the player select a difficulty.
- `requirements.txt` — keeps the project free of third-party dependencies.
- `test_mastermind.py` — automated tests for the required behaviour.

The difficulty configurations in this implementation are examples; the README requires difficulty modes but does not prescribe exact values. Review them before submission.

3. Test results

14 automated tests passed

The generated implementation passed all 14 tests in the test run.

Feedback logic	Passed
Repeated-symbol handling	Passed
Difficulty configurations	Passed
Invalid input and turn limits	Passed
Win, loss and quit handling	Passed

Testing limitation: I could inspect the public repository's source files and README, but I could not clone the repository in this environment because network access failed. These tests therefore validate the generated implementation, not a patch applied to your local repository. Run the tests on your Mac before treating the changes as final.

After extracting the ZIP, open Terminal in the project directory and run:

```
python3 -m unittest -v test_mastermind.py
python3 main.py
```

Play the game manually in all three difficulty modes. Test invalid guesses, use `h` to display history, and use `q` to quit.

4. Export this chat for your lab submission

Your README asks for the complete LLM chat-history link, while your lab handout also requests a chat-history document or PDF. SETAPESU26/... Lab_4_VibeCoding_Student_handout.pdf

Option A — Create the conversation link

1. Open this conversation in ChatGPT.
2. Click **Share** near the top of the conversation.
3. Create a shared link and copy it.
4. Include that link in your lab submission.

Check the shared conversation before submitting. A share link is accessible to anyone who obtains it, so do not include sensitive information.

Option B — Save a PDF copy

1. Open this conversation in your browser.
2. Press `Cmd + P` on your Mac.
3. Select **Save as PDF** as the destination.
4. Save the file as `Lab4_Chat_History.pdf`.

Check the PDF to ensure it contains the relevant conversation messages. Browser printing may omit some dynamic or collapsed content, so verify the saved document.

5. Final submission checklist

- [ ] 10-second video showing the original bug, recorded before editing.
- [ ] 10-second video showing the fixed game and new features.
- [ ] Updated code pushed to your personal GitHub fork.
- [ ] Separate Git commits for the four tasks.
- [ ] Complete LLM conversation link.
- [ ] Chat-history PDF or document in your `Lab–4` folder.

Next step: Extract the ZIP, run the tests, and review the changes. Then commit each task separately in your own repository. Do not submit the generated code without testing it locally.

‹/›

You said:

what all do i need to submit in the google classroom?, also give one final before vs after numbers to type and check for the recording

ChatGPT said:

1. What to submit in Google Classroom

According to your Lab 4 handout, create a `Lab–4` folder in your lab repository and include these deliverables. The handout specifically lists the before/after videos, updated code, and exported chat history. Lab_4_VibeCoding_Student_handout.pdf

#	Deliverable	What to include
1	Before video	10-second recording of the original game before any changes
2	After video	10-second recording showing the fixed game and new features
3	Updated code	Completed Python project, including your tests
4	Chat history	Exported ChatGPT/AI conversation as a PDF or document, plus the share link if available

Also ensure your code is pushed to your **personal GitHub repository**, with a separate commit for each task. Do not open a pull request to the original repository.

If Google Classroom has a specific submission field or format, follow the instructions posted there as well.

2. Exact numbers to type for the before/after recording

Use the same test in both recordings so the difference is easy to demonstrate.



Before — original game

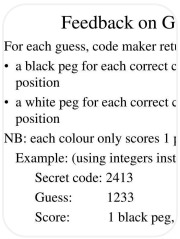
Record before editing

If the original game accepts four-digit guesses using digits `1 – 6`, enter:

1111

Copy guess

This is useful for testing whether repeated digits are incorrectly counted multiple times. Record the original feedback exactly as it appears.



After — fixed game

Record after testing

For a controlled test, use the secret code `1234` and enter:

1111

Copy guess

Expected feedback: **1 exact match, 0 partial matches.** Only one `1` exists in the secret code, so the other three guessed occurrences must not be counted.

Important: I cannot guarantee that `1111` exposes a visible bug in your original version without running that exact version. If the original game already handles duplicates correctly, record another genuine defect from the original code instead. Do not fabricate a before/after difference.

3. How to record each video

- **Before video (10 seconds):** launch the original game, enter the test guess, and show the feedback.
- **After video (10 seconds):** launch the corrected game, enter the same guess against the controlled secret code, and show the correct feedback.

For the after recording, you can run the automated tests beforehand:

```
python3 -m unittest -v test_mastermind.py
```

Then launch the game:

```
python3 main.py
```

The normal game randomly generates its secret code, so it may not generate `1234`. To demonstrate the exact controlled test, use the automated test output or temporarily configure the test instance with secret code `1234`; do not claim that the normal random game has that secret.

You said:

please give me the github cmds to fork push and commit,

ChatGPT said:

Follow these commands on your Mac to fork, clone, commit, and push your Mastermind Lab 4 project.

1. Fork the original repository

Open the original repository: SETAPESU26/20_mastermind ↗

1. Sign in to GitHub.
2. Click **Fork**.
3. Create the fork under your personal GitHub account.

2. Clone your fork

Replace `YOUR_GITHUB_USERNAME` with your actual GitHub username.

```
cd ~/Desktop

git clone https://github.com/YOUR_GITHUB_USERNAME/20_mastermind.git

cd 20_mastermind
```

Verify the remote:

```
git remote -v
git status
```

3. Add your updated code

If you downloaded my updated ZIP, extract it and copy the updated files into your cloned `20_mastermind` folder, replacing the corresponding files.

Make sure these files are included:

```
main.py
game.py
logic.py
requirements.txt
test_mastermind.py
```

Then run the tests:

```
python3 -m unittest -v test_mastermind.py
```

4. Commit each task separately

Stage only the files relevant to each task, then commit.

Commit 1 — Feedback bug

```
git add logic.py
git commit -m "Fix duplicate-aware feedback"
```

Commit 2 — Game lifecycle

```
git add game.py main.py
git commit -m "Complete game lifecycle"
```

Commit 3 — Difficulty modes

```
git add main.py game.py logic.py
git commit -m "Add difficulty modes"
```

Commit 4 — History and validation

```
git add main.py game.py logic.py
git commit -m "Improve guess history and input validation"
```

Important: These commands work as intended only if you make and verify each task's changes before creating its commit. If you copy all the completed files at once, the commits will not accurately represent separate tasks. For the assignment, it is better to make each change in sequence and commit it immediately after testing.

5. Push your commits to GitHub

Check your current branch:

```
git branch --show-current
```

If it is `main`, push using:

```
git push -u origin main
```

If the branch has another name, replace `main` with that name.

6. Final verification

```
git status
git log -4 --oneline
git remote -v
```

Confirm that:

- `git status` shows no uncommitted changes.
- Your commits appear in the log.
- `origin` points to your personal fork, not `SETAPESU26/20_mastermind`.
- Your code is visible on your personal GitHub repository.

Do not run `git push` against the original repository. Your submission should be in your personal fork, with separate commits for the required tasks.

You said:

thats all right, after mov, before and the chat link , and updated code

ChatGPT said:

Yes. Based on your Lab 4 handout, these are the items you need to submit in Google Classroom. Lab_4_VibeCoding_Student_handout.pdf

- **Before video** — 10-second recording of the original game before changes.
- **After video** — 10-second recording of the corrected game after changes.
- **Chat link** — shareable link to your ChatGPT conversation.
- **Updated code** — completed Python project pushed to your personal GitHub repository.

Also export your chat history as a PDF or document and include it in your `Lab–4` folder, as required by the handout.

Final check: Make sure your GitHub repository contains the updated code and separate commits for each task. Do not submit only the ZIP if your instructor expects the code in your personal GitHub repository.